Modelska analiza II
2024/25

10. naloga – Direktno reševanje Poissonove enačbe

Relaksacijske metode so primerne za reševanje splošne enačbe eliptičnega tipa na dokaj poljubnih definicijskih področjih in pri dokaj splošnih robnih pogojih. Kadar pa lahko prevedemo problem na eliptično enačbo s konstantnimi koeficienti (Poissonovo enačbo) in so meje področja koordinatne črte katerega od separabilnih koordinatnih sistemov, robni pogoji pa linearni, tedaj lahko uporabimo direktne metode, ki dajo eksaktno rešitev s končnim algoritmom. Take rešitve so seveda mnogo učinkovitejše.

Osnova direktnih algoritmov je Fourierova metoda, ki jo poznamo iz Matematične fizike in enačbo $\nabla^2 u(x,y) = g(x,y)$ reši v frekvenčnem prostoru. Za pravokotnik z robovoma a in b ter z robnimi pogoji I. vrste se rešitev glasi

$$u(x,y) = \sum A^{mn} \sin\frac{m\pi x}{a} \sin\frac{n\pi y}{b},$$
$$g(x,y) = \sum B^{mn} \sin\frac{m\pi x}{a} \sin\frac{n\pi y}{b}$$
$$A^{mn} = -B^{mn}/k_{mn}^2, \qquad k_{mn}^2 = \pi^2(m^2/a^2 + n^2/b^2).$$

Pri numeričnem reševanju seveda uporabimo algoritem FFT. Za običajni razvoj FFT v eksponentne valove moramo območje sicer v obeh smereh liho podaljšati, da dosežemo periodične robne pogoje. Zato je ceneje uporabiti hitro sinusno transformacijo

$$G^{mn} = \sum_{j=1}^{J-1}\sum_{\ell=1}^{L-1} g_{j\ell} \sin\frac{\pi j m}{J} \sin\frac{\pi \ell n}{L}.$$

Transformiranka rešitve se potem dobi kot

$$U^{mn} = \frac{h^2}{2}\frac{G^{mn}}{\cos\frac{\pi m}{J} + \cos\frac{\pi n}{L} - 2},$$

rešitev sama pa z obratno transformacijo

$$u_{j\ell} = \frac{2}{J}\frac{2}{L}\sum_{m=1}^{J-1}\sum_{n=1}^{L-1} U^{mn} \sin\frac{\pi j m}{J} \sin\frac{\pi \ell n}{L}.$$

Če je robni problem nehomogen, lahko predpisane robne vrednosti prevržemo na desno stran in jih pridružimo gostoti izvirov: $g_{J-1,\ell} \to g_{J-1,\ell} - f_\ell/h^2$.

Pri robnih pogojih druge vrste izhajamo seveda iz hitre kosinusne transformacije.

Za učinkovitost metode dvodimenzionalna Fourierova transformacija pravzaprav ni potrebna. Že če opravimo FFT v eni smeri, nam za delne transformiranke U_j^m preostane v vsakem stolpcu tridiagonalni sistem enačb, ki ga znamo rešiti[1] z zahtevnostjo reda J. Iz

$$u_{j-1,\ell} + u_{j+1,\ell} + u_{j,\ell-1} + u_{j,\ell+1} - 4u_{j\ell} = h^2 g_{j\ell}$$

sledi

$$U_{\ell-1}^m - U_\ell^m(4 - 2\cos\tfrac{m\pi}{J}) + U_{\ell+1}^m = h^2 G_\ell^m, \qquad m = 1, 2, \ldots, (J-1).$$

Vidimo, da bi bil operator za smer y (indeks ℓ) lahko celo splošnejši in tudi robni pogoj v tej smeri bi bil lahko poljubne linearne sorte, pa bi še vedno dobili tridiagonalni sistem enačb. Le Fourierova transformacija terja čisti operator $\partial^2/\partial x^2$ in robni pogoj prve ali druge vrste.

Na koncu naj omenimo še metodo ciklične redukcije, pri kateri shajamo brez Fourierove transformacije: diskretne enačbe eliptičnega operatorja predelamo z medsebojnimi eliminacijami na tridiagonalni sistem. Metoda je dokaj zapletena, nekaj malega o njej je v *Numerical Recipes*, vendar algoritem ni podan. Baje je najučinkovitejša kombinirana metoda FACR, ki združuje ciklično redukcijo in FFT.

[1] npr. podprogram `tridag` iz *Numerical Recipes* ali `scipy.linalg.solve_banded` v Pythonu

1. Določi poves v središču kvadratne opne, napete na vodoraven okvir, zaradi obtežbe, ki je enaka kot v 6. nalogi. Metodo lahko preiskusiš na neobteženi opni, kjer lahko uporabiš SOR, 2D FFT in 1D FFT.

2. Enakostraničen kovinski valj ($2r = h$) hladimo na zgornji in spodnji ploskvi na stalno temperaturo T_1, plašč valja pa na višjo temperaturo T_2. Določi temperaturni profil!

Sinusna transformacija obstaja v štirih tipih: na obeh robovih lahko liha simetrijska točka, v kateri je funkcija enaka nič, leži na polovici med prvo podano točko in zrcaljeno točko, ali pa v mrežni točki pred prvo podano točko (ničelna točka v nobenem primeru ni del podatkov). Sinusna transformacija je do normalizacijskega faktorja sama sebi inverz, pri čemer v primeru različne postavitve simetrijske točke na obeh robovih tipa robov zamenjamo. Podobno velja za kosinusno transformacijo. Podrobno dokumentacijo najdemo v dokumentaciji knjižnice `fftw3`. Python implementacijo najdete v `scipy.fft.dst`/`scipy.fft.idst`, kjer lahko izbirate tip transformacije.